

Proyecto Final de Procesamiento Digital de Señales

Transformador de imágenes para limpiar chroma y optimizar archivos antes de enviarlos a IA

Alejandro Apodaca Córdova — Matrícula m041852

Gael Calderón Robles — Matrícula m042449

Ingeniería Mecatrónica, CETYS Universidad, Campus Mexicali

3 de junio de 2026

Resumen

El proyecto consiste en una página web que toma una imagen original, detecta si tiene fondo de chroma y genera una versión limpia antes de enviarla a un chat o sistema de inteligencia artificial. La idea principal se resume así: *fondo limpio y archivo optimizado*. El sistema procesa la imagen en el navegador, estima el color de chroma, permite ajustarlo con un selector y separa dos decisiones con controles independientes: chroma y comprimir. Si se activa chroma, el usuario decide si el fondo queda transparente o se reemplaza por un color sólido. Si la salida conserva transparencia, se guarda PNG y no se presenta como compresión JPG. Si la salida usa color sólido, puede comprimirse como JPEG para ahorrar espacio. Aunque la interfaz final es simple, el funcionamiento interno aplica conceptos de procesamiento digital de señales: adquisición de una señal visual, segmentación por color, remuestreo espacial, codificación, medición de error y modelización del ahorro.

1 Problema de investigación

Muchas aplicaciones modernas permiten subir imágenes a modelos de IA. El problema es que una imagen tomada con un teléfono puede pesar varios megabytes y además puede contener un fondo innecesario. En casos de chroma key, por ejemplo, el fondo verde o azul no aporta información al objeto principal y puede distraer tanto al usuario como al sistema que recibe la imagen. Enviar imágenes pesadas o con fondo artificial aumenta el tiempo de carga, el ancho de banda y el tamaño de los datos si la imagen se convierte a base64 para transmitirse por una API.

Por eso se plantea el siguiente problema de investigación: **¿cómo limpiar el fondo chroma y reducir el peso de una imagen antes de enviarla a un sistema de IA, manteniendo una versión visualmente útil para el usuario?**

Las variables principales del problema son:

- Tamaño original de la imagen, medido en bytes.
- Tamaño de la imagen optimizada, medido en bytes.
- Porcentaje de ahorro.
- Dimensión máxima permitida después del remuestreo.
- Calidad JPEG usada para la compresión.
- Color de chroma detectado o ajustado por el usuario.
- Estado de los controles: quitar chroma activado/desactivado y compresión activada/desactivada.
- Modo de salida: fondo transparente o fondo con color sólido cuando se aplica chroma.
- Error entre la imagen procesada y la imagen codificada.

La relevancia del problema está en que optimizar imágenes antes de subirlas reduce recursos de red y mejora la experiencia de uso. En ingeniería, esto permite tomar una decisión técnica con datos y no solo con percepción visual.

2 Preguntas de investigación

Para guiar el proyecto se definieron tres preguntas:

1. ¿Cuánto peso se puede ahorrar al remuestrear una imagen y codificarla como JPEG?
2. ¿Cómo detectar y comunicar el color de chroma de forma simple?
3. ¿Qué limitaciones aparecen cuando se usa segmentación por color, transparencia PNG o compresión JPEG para preparar imágenes destinadas a sistemas de IA?

3 Marco teórico

Una imagen digital puede entenderse como una señal discreta bidimensional $x[m, n]$. Cada coordenada representa una muestra espacial y cada muestra contiene información de color. Cuando se reduce la resolución de la imagen, se reduce el número total de muestras. Este proceso se conoce como remuestreo espacial.

El prototipo también puede convertir la imagen a luminancia usando:

$$Y = 0.299R + 0.587G + 0.114B$$

Esta operación resume la información de color en brillo, lo cual puede ser útil cuando el color no es esencial para el análisis.

Para remover chroma, el sistema estima un color dominante en los bordes de la imagen. Después compara cada pixel contra ese color usando una distancia en el espacio RGB. Si la distancia es pequeña, el pixel se considera fondo. Si la distancia está cerca del umbral, se aplica una transición suave para evitar bordes recortados. El usuario también puede ajustar el color con un selector visual.

Cuando el control *chroma* está desactivado, el sistema solo reporta el color detectado y no altera el fondo. Cuando está activado, elimina los pixeles cercanos al color seleccionado. Si el modo es transparente, guarda PNG porque JPEG no puede guardar transparencia, y por eso no muestra un porcentaje de ahorro de JPG. Si el modo es color, reemplaza el fondo por un color sólido y puede guardar JPEG cuando *comprimir* está activo. El control *comprimir* es independiente: si está apagado, se conserva la resolución original y se oculta la métrica de ahorro; si está encendido y la salida no necesita transparencia, se aplica remuestreo y codificación para ahorrar espacio. JPEG usa compresión

con pérdida: reduce información para obtener un archivo más pequeño. Para medir el error se calcula el error cuadrático medio:

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

y la relación señal a ruido pico:

$$PSNR = 10 \log_{10} \left(\frac{255^2}{MSE} \right)$$

En la interfaz final no se muestra PSNR para no confundir al usuario, pero sí se calcula internamente y se usa como evidencia técnica en el reporte.

4 Herramienta moderna desarrollada

Se creó una aplicación web con Vite, React, TypeScript y Tailwind CSS. La herramienta funciona en el navegador, por lo que la imagen se transforma antes de llegar a un servidor. Esto es importante porque el objetivo del proyecto no es solo comprimir una imagen, sino evitar que una imagen innecesariamente pesada sea enviada a una aplicación de IA.

La interfaz final se diseñó para ser directa:

- El usuario sube una imagen.
- La página detecta el color de chroma si aparece.
- El usuario activa o desactiva chroma.
- El usuario elige si el fondo queda transparente o se reemplaza por color.
- El usuario activa o desactiva comprimir.
- Se muestran métricas de tamaño, ahorro de espacio cuando hay compresión JPG y color detectado.
- El usuario puede descargar la salida solo cuando hay ahorro o cuando se aplicó limpieza de chroma.

Como comparación reproducible fuera del navegador, el repositorio incluye el script `scripts/remove-background.py`. Este script usa Pillow en Python para hacer chroma key

por distancia de color, con los mismos dos modos conceptuales: transparencia o reemplazo por color sólido. No reemplaza al flujo web; sirve como evidencia técnica adicional de que el método también se puede ejecutar con herramientas conocidas de procesamiento de imágenes.



Figura 1: Interfaz web del transformador de imágenes.

5 Metodología

La metodología aplicada fue la siguiente:

1. Adquirir una imagen real desde el navegador mediante carga de archivo.
2. Decodificar la imagen y registrar sus dimensiones y bytes originales.
3. Estimar el color de chroma a partir de muestras en los bordes.
4. Si el control de chroma está activo, segmentar el fondo por distancia de color y aplicar borde suave.
5. Si el control de chroma está activo, quitar el fondo como transparencia o reemplazarlo por color sólido.

6. Si el control de compresión está activo y la salida no necesita transparencia, remuestrear la imagen para limitar su lado mayor a 1280 píxeles.
7. Codificar la imagen resultante como PNG si conserva transparencia, JPEG con calidad 72% si se comprime sin transparencia, o conservar la salida original cuando no hay una transformación útil.
8. Comparar el tamaño original contra el tamaño optimizado o limpio.
9. Calcular porcentaje de ahorro, expansión base64 evitada, MSE y PSNR.
10. Mostrar al usuario las métricas más importantes para la decisión: original, salida, ahorro y color de chroma.

Esta metodología define variables, aplica un método de investigación, ejecuta un experimento reproducible y permite interpretar los datos obtenidos.

6 Diseño experimental

Para validar el prototipo se usó una imagen de prueba reproducible incluida en el repositorio como `fixture-1.png`. Las variables controladas fueron:

- Calidad JPEG: 72%.
- Dimensión máxima: 1280 px.
- Formato de salida: PNG cuando hay transparencia, JPG cuando hay fondo sólido y se comprime.
- Controles independientes para chroma y compresión.
- Procesamiento local en navegador.

Las variables de respuesta fueron:

- Tamaño original.
- Tamaño optimizado.
- Porcentaje de ahorro.
- Relación de compresión.

- Base64 evitado.
- PSNR.
- Color de chroma detectado.

7 Resultados

Con la imagen de prueba, el sistema obtuvo los resultados de la Tabla 1. El caso demuestra que una imagen puede mantenerse útil visualmente y reducirse de forma considerable para transporte digital.

Cuadro 1: Resultados de la prueba reproducible con `fixture-1.png`.

Métrica	Resultado
Resolución original	1600 × 1000 px
Resolución optimizada	1280 × 800 px
Tamaño original	572 KB
Tamaño optimizado JPG	91 KB
Ahorro directo	84.1%
Relación de compresión	6.31x
Base64 evitado	642 KB
PSNR	36.2 dB

El patrón principal es claro: la reducción de resolución y la compresión JPEG disminuyen drásticamente el peso del archivo. En imágenes con chroma, la segmentación por color agrega una segunda utilidad: limpiar el fondo antes del transporte digital. Para un usuario no técnico, el porcentaje de ahorro y el color detectado son las métricas más importantes. Para justificarlo técnicamente, el reporte conserva las métricas de DSP y modelo de datos.

8 Predicción y modelización

Además de medir el tamaño final, se modeló la expansión base64 con la regla:

$$B_{64} = 4 \left\lceil \frac{B}{3} \right\rceil$$

donde B es el tamaño en bytes del archivo binario. Este modelo predice cuánto crece una imagen al serializarse para transporte textual en una API.

También se modeló el porcentaje de ahorro:

$$Ahorro = \frac{B_{original} - B_{optimizado}}{B_{original}} \times 100$$

Con este modelo la aplicación puede informar al usuario cuántos recursos se reducen antes de subir la imagen.

Si el resultado conserva transparencia, la interfaz no presenta el porcentaje de ahorro como compresión JPG porque la prioridad es conservar el canal alfa en PNG. Si el ahorro calculado es cero y tampoco se aplicó limpieza de chroma, la interfaz no ofrece descarga. Esto evita presentar como resultado un archivo que no mejora el original.

9 Limitaciones técnicas

El proyecto también reconoce límites importantes:

- JPEG no conserva transparencia; por eso una salida con canal alfa debe quedarse en PNG y no se reporta como compresión JPG.
- La detección de chroma funciona mejor cuando el fondo es relativamente uniforme y aparece cerca de los bordes.
- Si el objeto tiene colores parecidos al chroma, la segmentación puede eliminar partes del objeto.
- El helper de Python requiere Pillow instalado; por eso se documenta como alternativa reproducible y no como dependencia obligatoria de la web.
- La compresión puede afectar texto pequeño o detalles finos.
- PSNR mide diferencia numérica, pero no garantiza que un modelo de IA interprete exactamente igual la imagen.
- La compatibilidad de formatos depende del navegador.
- El modelo de unidades de contexto es una aproximación, no el conteo interno de un proveedor específico de IA.

Para reducir estas limitaciones se podrían agregar validaciones futuras como OCR para texto pequeño, SSIM para comparación perceptual y pruebas con modelos multimodales reales.

10 Conclusiones

El proyecto demuestra que aplicar procesamiento digital de señales a imágenes puede resolver un problema actual: limpiar fondos innecesarios y reducir el peso de archivos antes de enviarlos a sistemas de IA. Con una configuración simple de 1280 px y calidad JPEG de 72%, la prueba obtuvo un ahorro de 84.1%. Además, el flujo de chroma permite quitar el fondo como transparencia o reemplazarlo por color sólido. La interfaz final se mantuvo deliberadamente simple para que cualquier persona entienda que la transparencia se conserva en PNG y que la compresión JPG solo aplica cuando no se necesita canal alfa.

La conclusión principal es que la optimización local permite ahorrar recursos sin complicar la experiencia del usuario. El valor del proyecto está en unir una técnica de DSP con una necesidad moderna de aplicaciones web e inteligencia artificial.

11 Reflexión sobre la competencia evaluada

La competencia 24ICE04 se relaciona con investigar problemas complejos de ingeniería y obtener conclusiones válidas a partir de datos. En este proyecto se practicó esa competencia al definir un problema, controlar variables, medir resultados e interpretar el ahorro obtenido.

La competencia 24ICE05 se relaciona con crear, seleccionar, aplicar y reconocer limitaciones de herramientas modernas de ingeniería y TI. En este proyecto se aplicó al construir una herramienta web funcional, seleccionar tecnologías adecuadas y reconocer que la compresión JPEG no siempre es suficiente para todos los casos. La reflexión final es que una herramienta moderna no debe ser complicada para ser útil: debe resolver un problema real, mostrar datos claros y reconocer sus límites.

Publicación y repositorio

La página publicada del proyecto se encuentra en:

`https://capiz.apoapps.com`

El código fuente se encuentra en GitHub:

`https://github.com/apoapps/proyecto-final-procesamiento-digital-senales`

Referencias

- [1] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*. Pearson.
- [2] A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*. Pearson.
- [3] G. K. Wallace, "The JPEG Still Picture Compression Standard", *Communications of the ACM*.
- [4] MDN Web Docs, *Canvas API*.
- [5] Pillow Contributors, *Pillow Image Processing Library*.